



**COMSATS University Islamabad, Vehari Campus**  
**Mailsi Road, Vehari, Pakistan**

---

# **SOFTWARE REQUIREMENTS SPECIFICATION**

**(SRS DOCUMENT)**

**for**

**FYP Management System**  
Version 1.0

**By**

**Muhammad Tanveer** CIIT/FA22-BSE-027/VHR

*Supervisor*

**Qaizar Javed**

**Bachelor of Science in Computer Science (2022–2026)**

# Table of Contents

|    |                                                 |    |
|----|-------------------------------------------------|----|
|    | Revision History .....                          | 3  |
| 1. | Introduction .....                              | 4  |
|    | 1.1 Purpose .....                               | 4  |
|    | 1.2 Scope .....                                 | 4  |
| 2. | Overall Description .....                       | 5  |
|    | 2.1 Product Perspective .....                   | 5  |
|    | 2.2 Operating Environment .....                 | 6  |
|    | 2.3 Design and Implementation Constraints ..... | 7  |
| 3. | Requirement Identifying Technique .....         | 8  |
|    | 3.1 Use Case Diagram .....                      | 8  |
|    | 3.2 Use Case Description .....                  | 8  |
| 4. | Specific Requirements .....                     | 16 |
|    | 4.1 System Features .....                       | 16 |
| 5. | Quality Attributes .....                        | 17 |
|    | 5.1 Usability .....                             | 17 |
|    | 5.2 Performance .....                           | 18 |
|    | 5.3 Reliability .....                           | 18 |
|    | 5.4 Security .....                              | 18 |
|    | 5.5 Maintainability .....                       | 19 |
|    | 5.6 Scalability .....                           | 19 |
|    | 5.7 Summary .....                               | 19 |
| 6. | External Interface Requirements .....           | 19 |
|    | 6.1 User Interfaces .....                       | 19 |
|    | 6.2 Software Interfaces .....                   | 20 |
|    | 6.3 Hardware Interfaces .....                   | 20 |
|    | 6.4 Communication Interfaces .....              | 21 |
| 7. | Project Gantt Chart .....                       | 22 |
| 8. | References .....                                | 22 |

## Revision History

| No | Revision          | Changes Made                                               |
|----|-------------------|------------------------------------------------------------|
| 1  | Initial Draft     | Created initial SRS structure and added core requirements. |
| 2  | Supervisor Review | Updated modules and expanded requirement details.          |
| 3  | Formatting Fixes  | Improved readability and aligned document formatting.      |
| 4  | Refinement        | Updated diagrams and descriptions.                         |
| 5  | Final Approval    | Final SRS approved for submission.                         |

# **1 Introduction**

## **1.1 Purpose**

The purpose of this Software Requirements Specification is to define, clarify and document all functional and non functional requirements related to the development of the Final Year Project Management System. This system is designed to address the long standing challenges faced by academic institutions particularly COMSATS University in managing Final Year Projects across multiple departments and student batches. The purpose of the system is to provide a comprehensive, structured and automated platform that facilitates seamless coordination among students, supervisors, evaluation committees and administrative personnel.

The document outlines in detail how the system supports every stage of the Final Year Project life cycle including group formation, proposal submission, supervisor selection and approval, progress tracking, document uploads, midterm and final evaluations, and communication among stakeholders. By clearly defining requirements, the SRS acts as a reference for developers to implement the system accurately, for testers to validate system behavior, for maintainers to extend the platform in future semesters and for stakeholders to verify that their needs are fully addressed.

A central purpose of the system is to eliminate dependency on manual processes that cause communication gaps, evaluation inconsistencies, administrative delays and difficulty in maintaining proper records. The use of digital templates, structured evaluation rubrics and automated workflows ensures that all activities remain aligned with NCEAC requirements for standardization, fairness and transparency. The purpose of this SRS is to formally document the architecture, features, interactions, constraints and quality expectations necessary to deliver a professional and academically compliant platform.

This document also aims to define a unified and centralized approach for storing and retrieving project related information. Through role based access control, the system ensures that students, supervisors, committee members and administrators interact only with the functionalities and data that are relevant to their responsibilities. The purpose of the system extends further to include real time communication through notifications and messaging, which reduces delays and improves decision making. The SRS therefore provides a complete blueprint to implement a reliable, secure, scalable and user friendly system that enhances the academic and administrative experience of managing Final Year Projects.

## **1.2 Scope**

The scope of the Final Year Project Management System covers the complete digital transformation of the project management workflow used at COMSATS University for undergraduate degree programs. The system provides an integrated platform that supports all stakeholders involved in the Final Year Project process. Students can create groups, propose topics, submit documents, track progress and view evaluations. Supervisors can review proposals, approve or reject submissions, monitor group progress, provide comments, communicate with students and enter evaluation scores. Committee members can upload standardized templates, access submitted documents, assign evaluations and ensure compliance with accreditation standards. Administrators are responsible for managing user accounts, configuring system settings, monitoring project distribution and ensuring system

integrity throughout the semester.

The system falls within the scope of replacing traditional paper based and informal communication methods with a structured web based solution that maintains all records digitally. This includes proposals, progress reports, evaluation scores, presentation files and rubric based assessments. The system supports a full academic year of activities comprising both the fall and spring semesters. Within this scope, the system enforces NCEAC aligned evaluation rubrics, stores all documents in a centralized repository and ensures uniformity and fairness in how students and supervisors participate in the project.

The scope also includes the implementation of dashboards tailored to each role, providing clear navigation and access to tools necessary for carrying out responsibilities. The student dashboard focuses on group management and submissions, the supervisor dashboard emphasizes progress monitoring and feedback and the committee dashboard handles evaluations and template uploads. The admin dashboard supports system configuration, user creation and monitoring of supervisor workloads. These role specific interfaces ensure efficiency and reduce cognitive load for users.

Additionally the system scope extends to the communication layer which includes a messaging module based on polling. This enables continuous and reliable information exchange without requiring real time socket based infrastructure. Notifications ensure that users remain updated about deadlines, approvals, evaluations and required actions. The application is intended to run in modern browsers and use Django REST Framework as the backend, React as the frontend and SQLite as the development database, with scalability to enterprise level databases as needed.

The system's scope does not include unrelated academic modules, external attendance systems or external learning management systems. Instead, it focuses exclusively on delivering a comprehensive, efficient and transparent platform for managing Final Year Projects. It ensures that all stakeholders can interact smoothly, all processes follow structured workflows and all evaluation activities remain properly recorded and accessible for audits and future review. The system remains aligned with institutional policies and accreditation requirements, forming a dependable academic management tool for long term use.

## **2 Overall Description**

### **2.1 Product Perspective**

The Final Year Project Management System is designed as a centralized and fully integrated web based platform intended to modernize and automate the management of academic projects at COMSATS University. The system replaces traditional paper based documentation, manual communication and unstructured project workflows with a unified digital environment. It functions as an independent platform that supports multiple academic roles including students, supervisors, committee members and administrators through role based access control. Each user interacts with a customized interface that provides access only to functions that match their responsibilities, ensuring security, reliability and clarity in usage.

The product fits into the academic ecosystem as a core administrative tool that oversees the complete life cycle of a Final Year Project. From the initial stages of forming groups and

proposing topics to the final stages of evaluation and submission, the system provides a structured and trackable workflow. It removes the need for emails, paper forms, spreadsheets and manually maintained records which often lead to delays, inconsistencies and lost documentation. Instead all project related data is maintained in a single repository that ensures accuracy, permanence and easy retrieval.

The platform integrates multiple modules such as project selection, document submission, supervisor approval workflows, evaluation mechanisms, communication channels, and real time notifications. These modules work together within a three tier software architecture. The presentation layer is built using React and Tailwind CSS which ensures a responsive and user friendly interface. The application layer is developed using Django and the Django REST Framework which provide a secure and modular backend capable of enforcing business logic, user authentication, and data validation. The data layer uses SQLite for development and supports migration to PostgreSQL or other production ready relational databases when required.

In addition the system aligns directly with the guidelines of NCEAC which require standardized evaluation, equal supervision load distribution and proper archiving of student work. The product perspective includes supporting features for rubrics, templates, structured assessment forms and faculty workload tracking. These components position the system not only as a convenient administrative tool but also as an accreditation supportive platform that satisfies compliance requirements and quality assurance demands.

Overall the system serves as a foundational digital solution that brings automation, transparency and accountability into the Final Year Project process. It connects all stakeholders into a unified environment that simplifies collaboration, reduces confusion and increases academic efficiency.

## **2.2 Operating Environment**

The system is designed to run in a wide range of modern environments to support accessibility for students and faculty across various devices and locations. It is accessible through standard web browsers including Google Chrome, Mozilla Firefox, Microsoft Edge, Brave Browser and Safari. The platform does not require installation of any additional plugins, ensuring that users can access the system easily from university labs, personal laptops or mobile devices.

The backend is developed using Django and Django REST Framework which provide a secure, scalable and maintainable API layer. Django ensures reliable session management, form handling, query processing and role based authorization. The REST Framework facilitates communication between the frontend and backend using structured JSON based APIs. All server side logic including authentication, project approval workflows, evaluation rules, notification triggers, document handling and reporting is executed within the Django environment.

The frontend is built using React which enables smooth user experience, component based interface development and efficient rendering. Tailwind CSS is used to create a responsive and visually consistent interface that adapts to different screen sizes including laptops, desktops, tablets and mobile devices. The combination of React and Tailwind allows rapid development and ensures that complex student and faculty dashboards remain clear, organized and easy to navigate.

During development the database environment relies on SQLite due to its simplicity, low

setup requirements and suitability for initial prototyping. However, the system architecture is fully compatible with more advanced relational database systems such as PostgreSQL or MySQL which may be used in production environments to support larger user loads and long term archival storage. The system is hosted on a server that supports Python, Django and standard web protocols such as HTTP and HTTPS. Secure communication channels ensure that sensitive project related data such as evaluations, reports and supervisor feedback remain protected.

The operating environment also includes integration with the university network where required. Role based logins, controlled resource access and secure database connectivity ensure that the system can function reliably in both local server deployments and cloud based hosting solutions. The system is designed to work under varying network conditions and automatically handles API request failures through retry mechanisms and validation checks.

### **2.3 Design and Implementation Constraints**

The design of the Final Year Project Management System is influenced by several academic, technical and operational constraints that must be considered throughout development. One of the primary constraints is adherence to NCEAC standards. The system must implement evaluation procedures that match the assessment rubrics defined by the accreditation council. Each evaluation component including proposal review, midterm evaluation and final evaluation must follow consistent patterns that ensure fairness and transparency across all students and departments.

Role based access control is a required constraint to ensure privacy and data integrity. Students should not access supervisor or committee functionalities, supervisors should only access groups assigned to them and committee members must not view restricted administrative data. This constraint drives the need for detailed permission checking and strict authorization logic throughout the system.

The communication module is designed using polling instead of real time socket connections. This constraint is based on infrastructure limitations and the need to simplify server management. Although polling provides slightly slower updates compared to Web Socket technology, it remains sufficient for academic messaging between students and supervisors.

Storage and performance constraints also affect the system design. During development the use of SQLite restricts the number of simultaneous write operations and limits large data transactions. While suitable for development, this constraint requires careful design of database queries and indexing to ensure that the system remains responsive. Migration to PostgreSQL in production can address these limitations but requires compatibility during initial implementation.

Network policy constraints imposed by the university must be followed, including secure login sessions, handling of confidential project reports, and safeguarding evaluation data. The system must ensure protection against unauthorized access, data leakage and misuse of digital resources. Compliance with these policies influences password hashing, session management, database encryption considerations and audit trails for critical operations.

Overall these constraints guide the system toward a secure, standards compliant and maintainable architecture that aligns with institutional requirements and academic regulations.

### 3 Requirement Identifying Technique

#### 3.1 Use Case Diagram

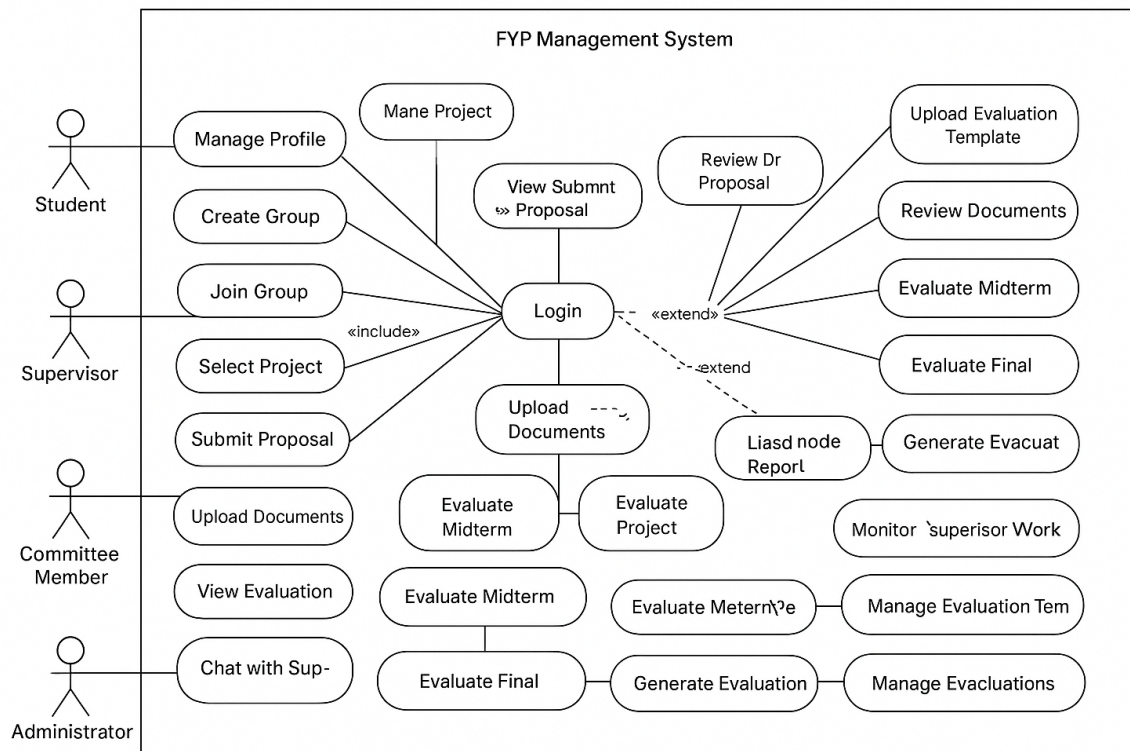


Figure 1: Use Case Diagram

#### 3.2 Use Case Description

##### Common Use Case: Login

**Actor(s):** Student,Supervisor,Committee Member,Administrator

**Preconditions:**

- User is registered in the system.

**Postconditions:**

- User is authenticated and redirected to their dashboard.

**Main Flow:**

1. User enters login credentials.
2. System validates the credentials.



3. System grants access and loads the appropriate role based interface.

**Extensions:**

- Incorrect credentials → System prompts for re-entry.

**Student Use Case: Manage Profile**

**Actor:** Student

**Preconditions:** Student is logged in.

**Postconditions:** Profile is updated and stored.

**Main Flow:**

1. Student opens the profile section.
2. Student updates personal or academic information.
3. System validates and saves changes.

**Student Use Case: Create Group**

**Actor:** Student

**Includes:** Manage Profile

**Preconditions:**

- Student is logged in.
- Student profile is complete.

**Postconditions:** A new group is created.

**Main Flow:**

1. Student selects “Create Group”.
2. System includes “Manage Profile” to verify completeness.
3. Student enters group details.
4. System saves the group and updates records.
5. System triggers “Receive Notifications” (extend).

### **Student Use Case: Join Group**

**Actor:** Student

**Preconditions:** Student is not already in a group.

**Postconditions:** Student joins an existing group.

**Main Flow:**

1. Student searches available groups.
2. System checks group capacity and eligibility.
3. Student confirms joining.
4. System updates records and notifies group members.

### **Student Use Case: Select Project**

**Actor:** Student

**Includes:** View Project List

**Preconditions:** Student belongs to a group.

**Postconditions:** Project is selected and stored.

**Main Flow:**

1. Student opens project selection area.
2. System includes “View Project List”.
3. Student selects a project.
4. System assigns and records the selection.

### **Student Use Case: Submit Proposal**

**Actor:** Student

**Includes:** Upload Documents **Extends:** Chat with Supervisor

**Preconditions:** A project is already selected.

**Postconditions:** Proposal is submitted.

**Main Flow:**

1. Student opens the proposal submission form.
2. System includes “Upload Documents”.
3. Student submits the proposal.
4. System confirms submission and triggers notifications.

### **Student Use Case: Upload Documents**

**Actor:** Student

**Extends:** Submit Revision

**Main Flow:**

1. Student uploads documents (proposal, report, etc.).
2. System validates file types and size.
3. System stores files securely.

### **Student Use Case: View Feedback**

**Actor:** Student

**Main Flow:**

1. Student views supervisor or committee feedback.
2. System retrieves stored comments and evaluation data.

### **Student Use Case: View Evaluation**

**Actor:** Student

**Main Flow:**

1. Student selects evaluation results.
2. System displays midterm, final or project evaluation results.

### **Student Use Case: Chat with Supervisor**

**Actor:** Student

**Extends:** Submit Proposal

**Main Flow:**

1. Student opens chat interface.
2. Student sends a message to the supervisor.
3. System stores the conversation and notifies the supervisor.

### **Student Use Case: Receive Notifications**

**Actor:** Student

**Extends:** Any action requiring alerts (group changes, feedback, evaluation)

**Main Flow:**

1. System generates notifications.
2. Student receives alerts on dashboard or email.

### **Supervisor Use Case: Review Proposal**

**Actor:** Supervisor

**Includes:** View Submitted Proposal

**Preconditions:** Student has submitted proposal.

**Postconditions:** Proposal reviewed.

**Main Flow:**

1. Supervisor opens proposal list.
2. System includes “View Submitted Proposal”.
3. Supervisor reviews content.

**Extensions:**

- Approve or Reject Proposal (extend)

### **Supervisor Use Case: Approve or Reject Proposal**

**Actor:** Supervisor

**Extends:** Review Proposal

**Main Flow:**

1. Supervisor chooses Approve or Reject.
2. System updates status and notifies students.

### **Supervisor Use Case: Track Student Progress**

**Actor:** Supervisor

**Postconditions:** Progress logged and feedback may be added.

**Main Flow:**

1. Supervisor views project timeline and submissions.
2. System displays milestones, tasks and student updates.

**Extension:**

- Provide Feedback (extend)

**Supervisor Use Case: Provide Feedback**

**Actor:** Supervisor

**Extends:** Track Student Progress

**Main Flow:**

1. Supervisor enters comments or improvement suggestions.
2. System stores feedback and notifies students.

**Supervisor Use Case: Evaluate Project**

**Actor:** Supervisor

**Includes:** Review Documents **Extends:** Chat with Students

**Main Flow:**

1. Supervisor opens evaluation form.
2. System includes “Review Documents”.
3. Supervisor assigns grades and comments.

**Committee Use Case: Upload Evaluation Template**

**Actor:** Committee Member

**Main Flow:**

1. Committee member selects upload template option.
2. System validates template format.
3. System stores the evaluation template.

**Committee Use Case: Review Documents**

**Actor:** Committee Member

**Main Flow:**

1. Committee member opens document list.
2. System displays student submissions for evaluation.

### **Committee Use Case: Evaluate Midterm**

**Actor:** Committee Member

**Includes:** Use Evaluation Template

**Main Flow:**

1. Committee member selects midterm evaluation.
2. System includes “Use Evaluation Template”.
3. Committee member enters scores and comments.

### **Committee Use Case: Evaluate Final**

**Actor:** Committee Member

**Includes:** Use Evaluation Template

**Main Flow:**

1. Committee member selects final evaluation.
2. System includes “Use Evaluation Template”.
3. Committee member records final evaluation results.

### **Committee Use Case: Generate Evaluation Report**

**Actor:** Committee Member

**Includes:** Evaluate Midterm, Evaluate Final

**Main Flow:**

1. Committee member generates a consolidated report.
2. System includes midterm and final evaluation data.
3. System outputs complete evaluation report.

### **Admin Use Case: Manage Users**

**Actor:** Administrator

**Main Flow:**

1. Admin adds, updates, or removes users.
2. System stores changes.

### **Admin Use Case: Assign Roles**

**Actor:** Administrator

**Includes:** Manage Users

**Main Flow:**

1. Admin selects user role assignments.
2. System includes “Manage Users”.
3. System updates role permissions.

### **Admin Use Case: Configure System Settings**

**Actor:** Administrator

**Main Flow:**

1. Admin configures evaluation settings, deadlines, and parameters.
2. System stores updated settings.

**Extension:**

- Manage Evaluation Templates (extend)

### **Admin Use Case: Manage Evaluation Templates**

**Actor:** Administrator

**Extends:** Configure System Settings

**Main Flow:**

1. Admin uploads or edits evaluation templates.
2. System validates and stores templates.

### **Admin Use Case: Monitor Supervisor Workload**

**Actor:** Administrator

**Main Flow:**

1. Admin views supervisor assignment reports.
2. System shows number of students per supervisor.

## 4 Specific Requirements

### 4.1 System Features

#### Feature: Student Module

The Student Module provides functionalities that allow students to manage their project-related activities efficiently. The key features include:

- **Create and Manage Profile:** Students can create a personal profile with relevant information such as name, contact details, academic program and project preferences. They can update their profile information as needed.
- **Create and Join Groups:** Students can create new project groups or join existing ones. The system ensures group eligibility rules, such as maximum group size and membership restrictions are enforced.
- **Submit Project Proposals:** Students can submit detailed project proposals including title, objectives, methodology and expected outcomes. The system stores the submissions and notifies the assigned supervisor.
- **Upload Documents and View Evaluations:** Students can upload project documents such as reports, presentations and code files. They can also view feedback and evaluation scores provided by supervisors and committee members.
- **Chat and Communication:** Students can communicate directly with their assigned supervisors through an integrated messaging system for guidance, clarification and progress discussions.

#### Feature: Supervisor Module

The Supervisor Module allows supervisors to monitor and guide student projects. Its key features include:

- **Approve Student Groups:** Supervisors can review group formation requests and approve or suggest changes according to project requirements.
- **Review Proposals and Progress Reports:** Supervisors evaluate project proposals and periodic progress reports, providing feedback and suggestions for improvements.
- **Provide Grading and Feedback:** Supervisors assign grades for assignments, proposals and project milestones and provide structured feedback to guide students.
- **Communicate Through Messaging:** Supervisors can interact with students using the builtin chat system to ensure timely guidance and address queries.
- **Track Student Performance:** Supervisors can view the history of student submissions, feedback and group activities to monitor progress effectively.



### Feature: Committee Module

The Committee Module supports project evaluation and ensures standardization of assessments. Its features include:

- **Upload Evaluation Templates:** Committee members can create and upload evaluation rubrics and templates for consistent grading of proposals and final submissions.
- **Review Project Documents:** Committee members can access submitted reports, presentations and other documents for thorough evaluation.
- **Enter Marks for Mid and Final Evaluations:** Committee members can record scores for each evaluation stage, ensuring transparency and accountability.
- **Generate Evaluation Reports:** The system provides aggregated evaluation reports to help supervisors and administrators track project outcomes.

### Feature: Admin Module

The Admin Module manages overall system configuration and user management. Key features include:

- **Manage Users and Roles:** Administrators can add, remove, and update user accounts, assign roles such as student, supervisor or committee member and manage permissions.
- **Configure System Settings:** Administrators can customize system parameters such as project deadlines, notification settings and evaluation criteria.
- **Monitor Project Assignment Workload:** Admins can monitor the number of projects assigned to supervisors and committees to ensure balanced workload distribution.
- **Audit and Log Management:** The system keeps track of user actions, document uploads and evaluations for accountability and security purposes.
- **System Maintenance and Backup:** Administrators can perform system maintenance, database backups and restore data when necessary to ensure smooth operation.

## 5 Quality Attributes

### 5.1 Usability

The system must provide a clean, responsive and intuitive interface. Users, including students, supervisors, committee members and administrators should be able to navigate the system easily with minimal training. The design must support accessibility, readability and consistent user experience across devices, including desktops, tablets and mobile devices.

The interface should include clear menus, buttons, and labels for all major functions such as creating or joining groups, submitting proposals, uploading documents, viewing feedback and evaluating projects. Context sensitive help and tooltips should be available where necessary to assist users in understanding system features. Error messages should be informative and guide the user towards corrective action, reducing confusion and minimizing mistakes.

Additionally the system should provide proper feedback for all user actions. For example, when a student submits a proposal or uploads a document, the system should display confirmation messages to assure the user that the action was successful. Consistency in design patterns, fonts, color, schemes and layout across all modules enhances the usability and overall satisfaction of users. Usability testing should be conducted periodically to identify potential bottlenecks or confusing interface elements and improve the overall experience.

## **5.2 Performance**

The system must handle multiple concurrent users effectively especially during peak submission periods when many students may be submitting proposals or uploading project documents simultaneously. The backend should be optimized for quick response times and the database queries must be efficient to ensure that retrieving student, group and project data does not lead to delays or timeouts.

Caching mechanisms can be implemented to reduce the load on the database for frequently accessed data such as lists of available projects, group members or supervisor assignments. Pagination and lazy loading techniques should be used for displaying large lists such as submitted proposals or feedback history to avoid long page load times. Performance metrics such as average response time, peak load handling and system throughput should be monitored continuously and necessary optimizations should be applied.

## **5.3 Reliability**

The system must be reliable and available, ensuring that users can perform critical tasks such as submitting proposals, uploading documents and providing feedback without interruptions. Data consistency is crucial; the system must prevent data loss or corruption during concurrent operations. Transaction management and proper error handling should be implemented to maintain integrity when multiple users are interacting with the system simultaneously.

Backup and recovery mechanisms should be in place to safeguard project records, submissions and evaluations. Regular automated backups along with the ability to restore data in case of accidental deletion or system failure, will increase user confidence and trust in the system. The system should also provide notifications in case of scheduled maintenance or unexpected downtime, minimizing disruption to users.

## **5.4 Security**

Security is a critical attribute of the system. User authentication and authorization must ensure that only authorized students, supervisors, committee members and administrators can access their respective functionalities. Passwords must be stored securely using hashing algorithms and sensitive data should be transmitted using secure protocols such as HTTPS.

The system should prevent unauthorized access to project documents, evaluations and private feedback. Role based access control should be enforced to ensure that users can only perform actions permitted by their role. Measures against common security threats, such as SQL injection, cross-site scripting (XSS) and cross-site request forgery (CSRF), should be implemented. Additionally, logging and monitoring of critical operations will help in identifying and mitigating potential security breaches.

## **5.5 Maintainability**

The system should be designed to allow easy maintenance and updates. Code should be modular, well documented, and follow consistent naming conventions to facilitate future enhancements and debugging. Separation of concerns, use of design patterns and adherence to coding standards improve maintainability and reduce the cost of system modifications.

Database schema should be designed to support future expansion such as adding new types of evaluations or feedback mechanisms. Automated tests should be included to validate system functionality after updates, ensuring that new features or bug fixes do not introduce regressions. Clear documentation for installation, deployment and troubleshooting will aid administrators in maintaining the system efficiently.

## **5.6 Scalability**

The system must be scalable to accommodate increasing numbers of students, supervisors and project submissions over time. It should support horizontal scaling, such as adding more servers or computing resources, without requiring significant changes to the architecture. Efficient database indexing and query optimization will help the system scale while maintaining performance.

The system should also be capable of integrating with additional modules in the future, such as advanced analytics for project evaluations, automated notifications or integration with institutional learning management systems. Scalability ensures that the system can continue to serve the university efficiently as the number of users grows.

## **5.7 Summary**

By focusing on these quality attributes usability, performance, reliability, security, maintainability, and scalability the Final Year Project Management System ensures a robust, user friendly, and professional environment for managing student projects. These attributes collectively contribute to the satisfaction of all stakeholders and the long term success of the system.

# **6 External Interface Requirements**

## **6.1 User Interfaces**

The system provides distinct, role based dashboards for students, supervisors, committee members and administrators to ensure that each user type has access to functionalities relevant to their responsibilities.

**Student Dashboard:** Allows students to manage profiles,create or join groups,submit project proposals,upload documents,view feedback and evaluations and communicate with supervisors.

**Supervisor Dashboard:** Enables supervisors to review student proposals,approve group formations,provide feedback and grading,track progress and communicate directly with students.

**Committee Dashboard:** Supports committee members in reviewing project submissions,uploading evaluation templates,entering evaluation marks and generating progress reports.

**Administrator Dashboard:** Allows administrators to manage users, assign roles,configure system settings,monitor project workload and oversee overall system operations.

The interface is implemented using React JS along with Tailwind CSS to provide a modern,responsive and accessible design.All dashboards are designed for **\*\*ease of navigation\*\***, **\*\*clarity of information\*\*** and **\*\*consistency across devices\*\***,including desktops,laptops and tablets. Interactive elements such as modals,notifications,tooltips and forms are implemented to guide users efficiently through tasks,reducing errors and enhancing usability.

## 6.2 Software Interfaces

The system relies on multiple software components for smooth operation:

- **Django REST Framework (DRF):** Provides RESTful APIs for communication between the frontend React JS interface and the backend server,enabling secure data exchange and CRUD operations for projects,groups,users and evaluations.
- **SQLite Database:** A lightweight relational database is used to store user data,project records,group information,feedback and evaluation results.SQLite ensures quick data retrieval and transactional integrity for concurrent operations.
- **Browser to Server Communication:** The frontend communicates with the backend using HTTP requests for general operations and polling for real time updates such as chat messages and notifications.This ensures that all users see up to date information without refreshing the page.

## 6.3 Hardware Interfaces

The system does not require specialized hardware and can run on standard personal computers,university lab machines,or laptops with basic specifications.However,to ensure smooth operation and optimal performance,the following minimal hardware requirements are recommended:

- **Processor:** Intel i3 or equivalent (quad-core processors are preferred for handling multiple concurrent users and real time notifications).
- **RAM:** 4GB minimum,although 8GB or more is recommended to allow smooth operation when multiple browser tabs or applications are running simultaneously.

- **Storage:** At least 100 MB for software installation, with additional space for storing project related documents, reports and backups. Large project groups may require additional storage to accommodate multiple submissions and evaluation records.
- **Display:** Standard monitor capable of 1024x768 resolution or higher. A higher resolution display is recommended for better visualization of dashboards, tables, Gantt charts and UML diagrams.
- **Network:** Reliable internet connection is mandatory for accessing dashboards, submitting proposals, uploading documents and using the integrated chat system. A stable connection ensures real time updates for notifications, feedback and evaluation scores.

No additional peripherals are strictly required, though optional devices such as printers may be useful for printing evaluation reports, project proposals, or submission receipts. The system has been designed to function efficiently on a wide range of standard hardware, minimizing the need for high end configurations.

To support scalability, the backend server hosting the system should ideally run on machines with higher specifications, such as multicore processors, at least 8GB of RAM and sufficient storage capacity to manage concurrent users, store project documents and maintain historical evaluation data. This ensures that the system can accommodate increasing numbers of students, supervisors, and committee members without performance degradation.

Additionally university lab machines should be configured with updated web browsers compatible with React JS and modern web standards, ensuring full accessibility to all interface features such as dashboards, notifications, chat modules and document management tools.

Overall the system has been optimized to function effectively on standard hardware while providing a responsive and reliable experience for all users, minimizing hardware constraints and promoting accessibility across multiple platforms.

## 6.4 Communication Interfaces

The system supports reliable communication between clients (web browsers) and the server using standard protocols and data formats:

- **HTTP and HTTPS:** Secure communication protocols are used to ensure data privacy and integrity between clients and the backend server. HTTPS is enforced for all sensitive operations, such as login, document upload and submission of evaluations.
- **JSON-based API Exchange:** All data exchanged between the frontend and backend is structured in JSON format, ensuring compatibility, readability and ease of parsing for web based clients.
- **Polling Mechanism:** The system uses polling to fetch real time updates for chat messages, notifications and evaluation status. This ensures that users are promptly informed about group activity, feedback, and supervisor messages without manual refreshes.
- **Error Handling and Timeouts:** Communication protocols include proper error handling and retry mechanisms to ensure reliability and consistent user experience in case of network interruptions.

## 7 Project Gantt Chart

|     |                           |                 | Fall Semester (2025) |     |     |     | Spring Semester (2026) |     |     |     |     |
|-----|---------------------------|-----------------|----------------------|-----|-----|-----|------------------------|-----|-----|-----|-----|
| No. | Phase                     | Duration        | Sep                  | Oct | Nov | Dec | Jan                    | Feb | Mar | Apr | May |
| 1   | Requirements Gathering    | 01 Sep – 10 Oct |                      |     |     |     |                        |     |     |     |     |
| 2   | Analysis & Design         | 11 Oct – 20 Nov |                      |     |     |     |                        |     |     |     |     |
| 3   | UI Design                 | 21 Nov – 25 Dec |                      |     |     |     |                        |     |     |     |     |
| 4   | Backend Setup             | 01 Dec – 20 Jan |                      |     |     |     |                        |     |     |     |     |
| 5   | Frontend Development      | 05 Jan – 20 Mar |                      |     |     |     |                        |     |     |     |     |
| 6   | Backend Development       | 01 Dec – 30 Mar |                      |     |     |     |                        |     |     |     |     |
| 7   | Integration & Testing     | 05 Mar – 10 Apr |                      |     |     |     |                        |     |     |     |     |
| 8   | Bug Fixing & Optimization | 11 Apr – 01 May |                      |     |     |     |                        |     |     |     |     |
| 9   | Final Submission          | 05 May – 20 May |                      |     |     |     |                        |     |     |     |     |

## 8 References

1. Sommerville Ian Software Engineering Tenth Edition Pearson
2. Pressman R S and Maxim B R Software Engineering A Practitioners Approach McGraw Hill
3. Zhang Y and Li W Web Based Project Management Tools A Review
4. Stark J Access Control Systems Identity Management and Trust Models Springer